

比特币协议的实际运作方式（转译）

2024-7-18 · marshall

本文转载自[How the Bitcoin protocol actually works — DDI \(michaelnielsen.org\)](https://michaelnielsen.org/ddi/how-the-bitcoin-protocol-actually-works/)，笔者对文章进行了翻译工作。

有成千上万篇文章声称要解释比特币这种点对点在线货币。其中大多数文章对底层加密协议的介绍都是一笔带过，忽略了许多细节。即使是那些深入研究比特币的文章，也往往忽略了一些关键点。这篇文章的目的是要以清晰易懂的方式解释比特币协议背后的主要思想。我们将从基本原理入手，从理论上对协议的工作原理有一个大致的了解，然后再深入到细枝末节，研究比特币交易中的原始数据。

以这种细致的方式理解协议是一项艰苦的工作。人们很容易将比特币视为既定事实，并投身于如何利用比特币致富，比特币是否是泡沫、比特币是否有一天会终结税收等等的猜测中。这很有趣，但严重限制了人们对比特币的理解。我们需要了解了比特币协议的细节，就可以收获到全新的视野，特别是，这些细节使得使用比特币创建新型金融工具（如智能合约）成为可能。反过来，新的金融工具又可以用来创建新的市场，实现新形式的人类集体行为。这就是乐趣所在！

这篇文章主要是解释比特币协议的基本原理。要理解这篇文章，你需要熟悉公钥密码学以及与之密切相关的数字签名概念。我还假设你熟悉加密哈希算法。这些都不难。大学数学或计算机科学大一课程就能讲授其基本思想。这些思想非常漂亮，所以如果你不熟悉它们，我建议你花几个小时来熟悉一下。

比特币的基础是密码学，这似乎令人惊讶。难道比特币不是一种货币，而是一种发送秘密信息的方式吗？事实上，比特币需要解决的问题主要是确保交易安全——确保人们不能相互盗窃，或相互假冒，等等。在原子世界里，我们通过锁、保险箱、签名和银行保险库等设备来实现安全。在比特世界里，我们通过密码学来实现这种安全。这就是为什么比特币的核心是一个加密协议。

为了更好地理解比特币，我将在这篇文章中一步步从零建立比特币。首先，我将解释一种非常简单的数字货币，它基于几乎显而易见的理念。为了与比特币区分开来，我们称这种货币为 Infocoin。当然，我们的第一版 Infocoin 会有很多不足之处，因此我们会对 Infocoin 进行多次迭代，每次迭代只引入一两个简单的新想法。经过几次这样的迭代之后，我们就会得到完整的比特币协议。我们将重塑比特币！

这种策略比我一次性解释整个比特币协议要慢。但是，虽然你可以通过这种一次性的解释理解比特币的机制，却很难理解比特币为什么要这样设计。一步步解释的优势在于，它能让我们更清晰地理解比特币的每个元素。

第一步：签署意向书

那么，我们该如何设计数字货币呢？

从表面上看，数字货币听起来是非常不可行的。假设某个人——我们姑且称她为爱丽丝——有一些数字货币，她想把它们花掉。如果爱丽丝可以使用一串比特作为货币，我们如何防止她重复使用同一个比特串，从而制造出无限量的货币呢？或者，如果我们能以某种方式解决这个问题，我们又该如何防止其他人伪造这样一串比特，并用它来从爱丽丝那里偷钱呢？

这些都是使用信息作为货币必须克服的众多问题之一。

作为Infocoin的第一个版本，让我们找到一种方式，让Alice可以使用一串比特作为（非常原始和不完整的）货币形式，在这种方式下，至少给她提供了一些防止伪造的保护。假设Alice想给另一个人Bob一个infocoin，为了做到这一点，Alice写下消息“我，Alice，正在给Bob一个infocoin”，然后她使用私有的加密密钥对消息进行数字签名，并向整个世界宣布签名的比特串。

（顺便说一下，我使用大写的“Infocoin”来指代协议和一般概念，使用小写的“infocoin”来指特定的货币单位。在比特币世界中，类似的用法很常见，尽管不是普遍的。）

这个原型数字货币并没有很让人惊艳，但它确实有一些优点。世界上的任何人（包括Bob）都可以使用Alice的公钥来验证Alice真的是签署了消息“我，Alice，正在给Bob一个infocoin”的人。没有其他人能创建那个比特串，所以Alice不能转过身来说“不，我不是想给Bob一个infocoin”。所以协议确立了Alice真的打算给Bob一个infocoin。同样的事实——没有其他人能组成这样的签名消息——也给了Alice一些有限的防止伪造的保护。当然，在Alice发布了她的消息之后，其他人可以复制这个消息，所以从某种意义上说，伪造是可能的。但从头开始是不可能的。这两个属性——确立Alice的意图和有限的防止伪造——是这个协议真正值得注意的特点。

我还没有（完全）说清楚在这个协议中数字货币是什么。为了明确这一点：它只是消息本身，即表示数字签名消息“我，Alice，正在给Bob一个infocoin”的比特串。稍后的协议将是类似的，因为我们的所有数字货币形式将只是越来越复杂的消息。

使用序列号使硬币唯一可识别

Infocoin的第一个版本的问题是Alice可以不断地向Bob发送相同的签名消息。假设Bob收到了十份签名消息“我，Alice，正在给Bob一个infocoin”。这是否意味着Alice向Bob发送了十个不同的infocoins？她的消息是否意外地被复制了？也许她试图欺骗Bob相信她给了他十个不同的infocoins，而消息只向世界证明了她打算转移一个infocoin。

我们想要的是使infocoins变得独特。它们需要一个标签或序列号。Alice将签署消息“我，Alice，正在给Bob一个infocoin，序列号为8740348”。然后，稍后，Alice可以签署消息“我，Alice，正在给Bob一个infocoin，序列号为8770431”，Bob（和其他人）将知道正在转移一个不同的infocoin。

为了使这个方案有效，我们需要一个受信任的infocoin序列号来源。创建这样一个来源的一种方式引入一个银行。这个银行将为infocoins提供序列号，跟踪谁有哪个infocoin，并验证交易是否真的合法，

在更详细的情况下，假设Alice走进银行，说“我想从我的账户中提取一个infocoin”。银行从她的账户余额中减去一个infocoin，并为她分配一个新的、以前从未使用过的序列号，比如说1234567。然后，当Alice想把她的infocoin转给Bob时，她签署消息“我，Alice，正在给Bob一个infocoin，序列号为1234567”。但Bob不仅仅接受这个infocoin。相反，他联系银行，并验证：(a)那个序列号的infocoin属于Alice；(b)Alice还没有花掉这个infocoin。如果这两件事都是真的，那么Bob告诉银行他想接受这个infocoin，银行更新他们的记录，显示那个序列号的infocoin现在在Bob的拥有中，不再属于Alice。

让每个人都成为银行

这个最后的解决方案看起来很不错。然而，事实证明我们可以做一些更大胆的事情。我们可以完全从协议中消除银行。这在很大程度上改变了货币的性质。这意味着不再有任何单一的组织负责货币。当你考虑到中央银行拥有的巨大权力——对货币供应的控制——这是一个相当大的变化。

这个想法是让每个人（集体）成为银行。特别是，我们假设每个使用Infocoin的人都保留着一个完整的记录，显示哪些infocoins属于哪个人。你可以将这想象成一个显示所有Infocoin交易的共享公共分类账。我们将把这个分类账称为区块链，因为在比特币中，一旦我们到达那里，完整的记录将被称为区块链。

现在，假设Alice想把一个infocoin转给Bob。她签署消息“我，Alice，正在给Bob一个infocoin，序列号为1234567”，并将签名的消息交给Bob。Bob可以使用他的区块链副本来检查，确实，这个infocoin是Alice给的。如果检查通过，那么他将Alice的消息和他的交易接受广播到整个网络，每个人都更新他们的区块链副本。

我们仍然有“序列号来自哪里”的问题，但事实证明这很容易解决，所以我将在稍后讨论。一个更具挑战性的问题是这样的协议允许Alice通过双重支付她的infocoin来作弊。她发送签名消息“我，Alice，正在给Bob一个infocoin，序列号为1234567”给Bob，消息“我，Alice，正在给Charlie一个infocoin，序列号为[相同]1234567”给Charlie。Bob和Charlie都使用他们的区块链副本来验证那个infocoin确实是Alice的。如果他们几乎同时进行这种验证（在他们有机会彼此交流之前），他们都会找到，是的，区块链显示这个硬币属于Alice。所以他们都会接受交易，并且也会广播他们接受交易的声明。现在有一个问题。其他人应该如何更新他们的区块链？可能没有办法实现一个一致的共享交易分类账。即使每个人都能就一种一致的方式来更新他们的区块链达成一致，仍然存在一个问题，那就是要么Bob要么Charlie会被欺骗。

起初，双重支付对Alice来说似乎很难实施。毕竟，如果Alice先把消息发给Bob，然后Bob可以验证消息，然后告诉网络上的其他人（包括Charlie）更新他们的区块链。一旦这样做了，Charlie就不会再被Alice欺骗了。所以Alice可能只有一个短暂的时间段可以双重支付。然而，显然我们不希望有一个时间段。更糟糕的是，Alice可以使用一些技术来延长这个时间段。例如，她可以使用网络流量分析来找到Bob和Charlie可能有大量通信延迟的时候。或者她可能做一些事情故意干扰他们的通信。如果她能稍微减慢通信，那会使她的双重支付任务变得更容易。

我们如何解决双重支付问题？显而易见的解决方案是当Alice向Bob发送一个infocoin时，Bob不应该试图单独验证交易。相反，他应该将可能的交易广播到整个Infocoin用户网络，并要求他们帮助确定交易是否合法。如果他们集体决定交易没问题，那么Bob可以接受infocoin，每个人都会更新他们的区块链。这种类型的协议可以帮助防止双重支付，因为如果Alice试图用Bob和Charlie都花掉她的infocoin，网络上的其他人会注意到，网络用户会告诉Bob和Charlie交易有问题，交易不应该进行。

让我们更详细地假设Alice想给Bob一个infocoin。像以前一样，她签署消息“我，Alice，正在给Bob一个infocoin，序列号为1234567”，并将签名的消息交给Bob。像以前一样，Bob做一个基本检查，使用他的区块链副本来检查，确实，这个硬币目前属于Alice。但在那一点上，协议被修改了。Bob不仅仅接受交易。相反，他将Alice的消息广播到整个网络。网络的其他成员检查Alice是否拥有那个infocoin。如果是这样，他们广播消息“是的，Alice拥有infocoin 1234567，现在可以转给Bob。”一旦足够多的人广播了那个消息，每个人都更新他们的区块链，显示infocoin 1234567现在属于Bob，交易完成。

这个协议目前有很多不精确的元素。例如，“一旦足够多的人广播了那个消息”是什么意思？在这里“足够”到底意味着什么？它不能意味着网络上的每个人，因为我们事先不知道谁在Infocoin网络上。出于同样的原因，它不能意味着网络上一些固定比例的用户。我们现在不会立即给出解释。相反，在下一节中我将指出这种方法的一个严重问题。解决这个问题的同时，我也会给出针对上述问题的解释。

工作量证明

假设Alice想在我刚刚描述的基于网络的协议中双重支付。她可以通过接管Infocoin网络来做到这一点。让我们假设她使用一个自动化系统在Infocoin网络上设置了大量的独立身份，比如说十亿个，然后她试图用Bob和Charlie双重支付同一个infocoin。但是当Bob和Charlie要求网络验证他们各自的交易时，Alice的傀儡身份淹没了网络，向Bob宣布他们已经验证了他的交易，向Charlie宣布他们已经验证了他的交易，可能欺骗了一个或两个人接受交易。

有一个巧妙的方法可以避免这个问题，使用一个叫做工作量证明的概念。这个想法涉及两个方面：(1) 人为地使网络用户验证交易变得计算上昂贵；(2) 奖励尝试帮助验证交易的行为。使验证交易变得昂贵的好处是，验证不再受某人控制的网络身份数量的影响，而只受他们能够用于验证的总计算能力的影响。正如我们将看到的，通过一些巧妙的设计，我们可以使得作弊者需要巨大的计算资源来作弊，使其变得不切实际。

这就是工作量证明的要点。但要真正理解工作量证明，我们需要详细展开了解。

假设Alice向网络广播消息“我，Alice，正在给Bob一个infocoin，序列号为1234567”。

当网络上的其他人听到这个消息时，每个人都将其添加到他们被告知但尚未被网络批准的待处理交易队列中。例如，另一个网络用户David可能拥有以下待处理交易队列：

- 我，Tom，正在给Sue一个infocoin，序列号为1201174。
- 我，Sydney，正在给Cynthia一个infocoin，序列号为1295618。
- 我，Alice，正在给Bob一个infocoin，序列号为1234567。

David检查他自己的区块链副本，并且可以看到每笔交易都是有效的。他愿意通过向整个网络广播该有效性的消息来提供帮助。

然而，在这样做之前，作为验证协议的一部分，David需要解决一个难题——工作量证明。如果没有这个难题的解决方案，其余的网络就不会接受他对交易的验证。

David需要解决的难题是什么？为了解释这一点，我们需要引入一个固定的哈希函数，网络中的每个人都知道——它内置在协议中。比特币使用的是众所周知的SHA-256哈希函数，但任何加密安全的哈希函数都可以。让我们给David的待处理交易队列一个标签，这样它就有了一个我们可以引用的名称。假设David将一个数字（称为 *nonce*）附加到并哈希这个组合。例如，如果我们使用“Hello, world!”（显然这不是交易列表，只是用于说明目的的字符串）和 *nonce* 那么得到输出（输出为十六进制）

```
h("Hello, world!0") =  
1312af178c253f84028d480a6adc1e25e81caa44c749ec81976192e2ec934c64
```

David必须解决的难题——工作量证明——是要找到一个 *nonce*，使得当我们将附加到并哈希这个组合时，输出的哈希以一长串零开始。可以通过改变所需零的个数来调整这个难题的难度。一个相对简单的工作量证明难题可能只需要哈希开头的三到四个零，而一个更困难的工作量证明难题可能需要更长的零序列，比如说15个连续的零。上述找到的 *nonce*=0 是失败的，因为输出根本没有以零开头。尝试 *nonce*=1也不起作用：

```
h("Hello, world!1") =  
e9afc424b79e4f6ab42d99c81156d3a17228d6e1eef4139be78e948a9332a7d8
```

我们可以继续尝试不同的 *nonce* 值。最后，在 *nonce* =4250 我们得到：

```
h("Hello, world!4250") =  
0000c3af42fc31103f1fdc0151fa747ff87349a4714df7cc52ea464e12dcd4e9
```


这个 nonce 给我们提供了一个在哈希输出开始处有四个零的字符串。这足以解决一个简单的工作量证明难题，但不足以解决一个更困难的工作量证明难题。

使这个难题难以解决的原因是，来自密码哈希函数的输出表现得像一个随机数：即使输入改变一点点，哈希函数的输出也会完全改变。所以如果我们想要输出哈希值以10个零开始，那么David平均需要尝试 $16^{10} \approx 10^{12}$ 不同的值 才能找到一个合适的 nonce 。这是一个相当具有挑战性的任务，需要大量的计算能力。

显然，我们可以通过在哈希函数的输出中要求更多或更少的零来使这个难题更容易或更难。事实上，比特币协议通过对上述描述的工作量证明难题进行轻微的变体，获得了对难题难度的相当精细的控制。比特币的工作量证明难题不是要求前导零，而是要求一个区块的头部哈希值低于或等于一个称为目标的数字。这个目标会自动调整，以确保比特币区块平均大约需要十分钟来验证。

（在实践中，验证一个区块所需的时间有很大的随机性——有时一个新区块在一两分钟内就被验证了，其他时候可能需要20分钟甚至更长时间。可以简单地修改比特币协议，使验证时间更尖锐地集中在十分钟左右）

好的，假设David幸运地找到了一个合适的 nonce 。庆祝吧！（他将因为找到 nonce 而得到奖励，如下所述）。他将他批准的交易区块广播到网络，连同 nonce 值。Infocoin网络的其他参与者可以验证该 nonce 是工作量证明难题的有效解决方案。然后他们更新他们的区块链以包含新的交易区块。

如果工作量证明的想法要成功，网络用户需要有动机帮助验证交易。如果没有这样的动机，他们就没有理由花费宝贵的计算能力，仅仅帮助验证其他人的交易。如果网络用户不愿意花费这种力量，那么整个系统就不会工作。解决这个问题的方法是奖励帮助验证交易的人。特别是，假设我们奖励成功验证交易区块的人一些infocoins。只要infocoin奖励足够大，这将给他们参与验证的动机。

在比特币协议中，这个验证过程被称为挖矿。对于每个被验证的交易区块，成功的矿工会收到比特币奖励。最初，这被设置为50个比特币奖励。但是，对于每210,000个被验证的区块（大约每四年一次），奖励减半。到目前为止，这种情况只发生过一次，所以目前挖掘一个区块的奖励是25个比特币。这种采矿奖励的减半将每四年继续一次，直到2140年。到那时，挖掘一个区块的奖励将降至以下比特币。比特币实际上是比特币的最小单位，称为_satoshi_。所以在2140年，比特币的总供应量将停止增加。然而，这并不会消除帮助验证交易的动机。比特币还允许在交易中留出一些货币作为交易费，这将归帮助验证它的矿工所有。在比特币早期，交易费大多设置为零，但随着比特币的普及，交易费逐渐上升，现在是一个重要的额外激励。

你可以将工作量证明看作是批准交易的竞争。比赛中的每个条目都需要一点计算能力。矿工赢得比赛的机会（大致上，有一些例外）等于他们控制的总计算能力的份额。所以，例如，如果一个矿工控制了用于验证比特币交易的1%的计算能力，那么他们大致上有1%的机会赢得比赛。因此，只要大量的计算能力被用于竞争，不诚实的矿工就只可能有一个相对较小的机会来破坏验证过程，除非他们投入大量的计算资源。

当然，虽然令人鼓舞的是，不诚实的一方只有相对较小的机会来破坏区块链，但这还不足以让我们对货币有信心。特别是，我们还没有最终解决双重支付的问题。

我很快就会分析双重支付。在此之前，我想补充一个关于Infocoin描述的重要细节。我们希望Infocoin网络能够就交易发生顺序达成一致。如果我们没有这样的顺序，那么在任何给定时刻，可能还不清楚谁拥有哪些infocoins。为了帮助做到这一点，我们需要要求新区块总是包括指向链中最后验证的区块的指针，除了区块中的交易列表。（这个指针实际上只是前一个区块的哈希值）。所以通常区块链只是一系列交易区块的线性链，一个接一个，后面的区块每个都包含指向紧接前一个区块的指针：

原图暂不可用（原站链接失效）：[block_chain.png](#)

偶尔，区块链中会出现分叉。这可能发生，例如，如果碰巧两个矿工几乎同时验证了一个交易区块——他们都向网络广播了他们新验证的区块，有些人更新了他们的区块链的一种方式，其他人更新了另一种方式：

原图暂不可用（原站链接失效）：[block_chain_fork.png](#)

这造成了我们极力避免的问题——即不再清楚交易发生的顺序，可能也不清楚谁拥有哪些infocoins。幸运的是，有一个简单的想法可以用来消除任何分叉。规则是这样的：如果发生分叉，网络上的人会跟踪两个分叉。但在任何给定时间，矿工只工作于他们区块链副本中最长的分叉（最长链原则）。

假设，例如，我们有一个分叉，一些矿工首先收到区块A，一些矿工首先收到区块B。那些首先收到区块A的矿工会继续沿着那个分叉挖掘，而其他人则会沿着分叉B挖掘。让我们假设在分叉B工作的矿工是下一个成功挖掘区块的人：

原图暂不可用（原站链接失效）：[block_chain_extended.png](#)

在他们收到这个消息后，分叉A上工作的矿工会注意到分叉B现在更长，并将切换到那个分叉。很快，分叉A上的工作就会停止，每个人都将在同一条线性链上工作，区块A可以被忽略。当然，A中的任何待处理交易仍然会在分叉B上工作的矿工的队列中挂起，所以所有交易最终都会被验证。

同样，如果分叉A上工作的矿工是第一个扩展他们分叉的人。在这种情况下，分叉B上的工作将很快停止，我们再次拥有一个单一的线性链。

无论结果如何，这个过程确保了区块链有一个公认的区块时间顺序。在真正的比特币中，交易被认为不是确认的，直到：(1) 它是最长分叉中的一个区块的一部分，(2) 最长分叉中至少有5个区块跟随它。在这种情况下，我们说交易有“6个确认”。这给了网络时间来就区块的顺序达成共识。我们也将为Infocoin使用这个策略。

现在，让我们重新思考如果一个不诚实的人尝试双重支付会发生什么。假设Alice试图与Bob和Charlie双重支付。她可能采取的一种方法是尝试验证一个包含两项交易的区块。假设她拥有1%的计算能力，她偶尔会很幸运地通过解决工作量证明来验证区块。不幸的是，Alice几乎不可能在两条链上同时挖出新区块，尽管解决了工作量证明问题，对于Alice来说，双重支付会立即被Infocoin网络上的其他人发现并被拒绝。

一个更严重的问题是，如果她广播了两项分别与Bob和Charlie支付同一infocoin的独立交易。她可能，例如，向一组矿工广播一项交易，向另一组矿工广播另一项交易，希望这样两项交易都能得到验证。幸运的是，在这种情况下，正如我们所看到的，网络最终会确认其中一项交易，但不是两项。所以，例如，Bob的交易最终可能会被确认，这样Bob就可以自信地继续。与此同时，Charlie会看到他的交易没有被确认，因此会拒绝Alice的提议。所以这也不是一个问题。

双重支付的一个重要变体是如果Alice = Bob，即，Alice试图与Charlie花费她同时也在“花费”（即，给自己）的硬币。在这种情况下，Alice的策略是等待Charlie接受infocoin，这发生在交易在最长链中被确认6次之后。然后她将尝试在与Charlie的交易之前分叉链，添加一个区块，其中包括她支付给自己的交易：

原图暂不可用（原站链接失效）：[block_chain_cheating.png](#)

很遗憾，对于Alice来说，现在她很难赶上更长的分叉。其他矿工不会想帮助她，因为他们将在更长的分叉上工作。除非Alice能够解决工作量证明至少和其他网络上的人一样快——粗略地说，这意味着控制超过50%的计算能力——否则她只会越来越落后。当然，她可能会很幸运。例如，我们可以想象这样一种情况，Alice控制了1%的计算能力，但碰巧连续找到了六个额外的区块，而在网络的其余部分没有找到任何额外的区块。在这种情况下，她可能能够领先，并控制区块链。但是，这种特定事件的发生概率非常小。更一般的分析表明，Alice赶上来的概率是微不足道的，只有 $\frac{1}{100^6} = 10^{-12}$ ，除非她能够以接近所有其他矿工的速率解决工作量证明难题。

工作量证明和挖掘思想引发了许多问题。奖励多少才足以说服人们挖掘？Infocoin供应的变化如何影响Infocoin经济？Infocoin挖掘最终会集中在少数人手中，还是许多人？如果只是少数人，那不是危及系统的安全吗？假设交易费最终会达到平衡——这会不会使小额交易变得不那么有吸引力吗？这些都是很好的问题，但超出了这篇文章的范围。我可能会在未来的文章中回到这些问题（在比特币的背景下）。现在，我们将专注于理解比特币协议的工作原理。

比特币

让我们抛开Infocoin，描述实际的比特币协议。这里有一些新的想法，但除了下面讨论的一个例外，它们大多是对Infocoin的修改。

要在实践中使用比特币，你首先需要在你的电脑上安装一个钱包程序。为了让你了解这意味着什么，这里有一个叫做Multibit的钱包的截图。你可以看到左侧的比特币余额——0.06555555比特币，或者在我拍摄这个截图的那一天的汇率约为70美元——以及右侧的两笔最近交易，这些交易存入了那些0.06555555比特币：

原图暂不可用（原站链接失效）：[wallet_transaction.jpg](#)

假设你是一个商家，已经设立了一个在线商店，并决定允许人们使用比特币支付。你所需要做的就是告诉你的钱包程序生成一个比特币地址。作为回应，它将生成一个公钥/私钥对，然后哈希公钥以形成你的比特币地址：

原图暂不可用（原站链接失效）：[bitcoin_address.jpg](#)

然后你将你的比特币地址发送给想要购买商品的人。你可以通过电子邮件发送，甚至可以将地址公开放在网页上。这是安全的，因为地址只是你的公钥的哈希值，公钥本来就可以被世界知道。（我稍后会回到为什么比特币地址是哈希值，而不仅仅是公钥的问题。）

将要付比特币给你的人会生成一个交易。让我们看看一个实际交易的数据。下面显示的几乎就是原始数据。它在三个方面有所改变：(1) 数据已经被反序列化；(2) 为了参考方便，添加了行号；(3) 我缩写各种哈希和公钥，只放入了每个的前六个十六进制数字，而实际上它们要长得多。这里是数据：

```
1  {"hash":"7c4025...",
2   "ver":1,
3   "vin_sz":1,
4   "vout_sz":1,
5   "lock_time":0,
6   "size":224,
7   "in":[
8     {"prev_out":
9       {"hash":"2007ae...",
10        "n":0},
11     "scriptSig":"304502... 042b2d..."}],
12  "out":[
13    {"value":"0.31900000",
14     "scriptPubKey":"OP_DUP OP_HASH160 a7db6f OP_EQUALVERIFY OP_CHECKSIG"}]
```

让我们逐行进行讲解。

第1行包含交易其余部分的哈希值，7c4025...，以十六进制表示。这被用作交易的标识符。

第2行告诉我们这是比特币协议版本1的交易。

第3行和第4行告诉我们交易有一个输入和一个输出。我稍后会讨论具有更多输入和输出的交易，以及为什么这很有用。

第5行包含lock_time的值，可以用来控制交易何时最终确定。对于当今进行的大多数比特币交易来说，lock_time设置为0，这意味着交易立即最终确定。

第6行告诉我们交易的大小（以字节为单位）。注意，这不是被转移的货币金额！那会在稍后提到。

第7行到第11行定义了交易的输入。特别是，第8行到第10行告诉我们输入将从具有给定hash的早期交易的输出中取出，以十六进制表示为2007ae...。n=0告诉我们它将是那笔交易的第一个输出；我们很快就会看到一笔交易的多个输出（和输入）是如何工作的，所以现在不用担心这个。第11行包含发送货币的人的签名，304502...，后面是一个空格，然后是相应的公钥，04b2d...。这些都是十六进制的。

关于输入的一个值得注意的事情是，这里没有明确指定应该在这笔交易中花费多少比特币。事实上，前一笔交易的n=0输出中的所有比特币都被这笔交易花费了。所以，例如，如果早期交易的n=0输出是2个比特币，那么这笔交易将花费2个比特币。这看起来像是一个不方便的限制——就像试图用20美元的钞票买面包，却不能找零。解决方案当然就是使用具有多个输入和输出的交易机制来提供找零。我们将在下一节中讨论这一点。

第12行到第14行定义了交易的输出。特别是，第13行告诉我们输出的价值，0.319比特币。第14行有点复杂。主要要注意的是字符串a7db6f...是资金预期接收者的比特币地址（以十六进制书写）。事实上，第14行实际上是比特币脚本语言中的一个表达式。我现在不会详细描述那种语言，现在重要的是只需要知道a7db6f...是比特币地址。

你现在可以看到，通过这种方式，比特币解决了我在上一节中回避的问题：比特币序列号从哪里来？事实上，在比特币中，序列号是根据哈希值得到的。在上面的交易中，例如，接收者收到了0.319比特币，这些比特币来自具有哈希2007ae...（第9行）的早期交易的第一个输出。如果你去区块链中查找那笔交易，你会看到它的输出来自更早的交易。

使用交易哈希而不是序列号有两个巧妙之处。首先，在比特币中实际上没有任何单独的、持久的“硬币”，只有区块链中的一系列交易。意味着你不需要持久的硬币，只需要一个交易账本，这是一个聪明的想法。其次，通过这种方式操作，我们消除了任何中央机构发行序列号的需要。相反，序列号可以通过哈希交易自动生成。

事实上，你可以继续追溯交易链的历史。这个过程最终会停止。这可以通过两种方式之一发生。第一种可能性是，你将到达所谓的创世区块中的第一个比特币交易。这是一个特殊的交易，没有输入，但有一个50比特币的输出。换句话说，这笔交易建立了初始货币供应。创世区块被比特币客户端特别处理，我不会在这里详细介绍，尽管它与上面的交易类似。

追溯交易链的第二种可能性是，你最终会到达所谓的coinbase交易。除了创世区块，区块链中的每个交易区块都以一个特殊的coinbase交易开始。这是奖励验证该区块交易的矿工的交易。它使用类似但不完全相同的格式，如上面的交易。我不会详细介绍格式，但如果你想看一个例子，可以点击[这里](#)。你可以在[这里](#)阅读有关coinbase交易的更多信息。

我在上文没有说清楚的是，第 11 行的数字签名到底要签署什么。显而易见，付款人应该签署整个交易（除了交易哈希值，当然，哈希值必须在稍后生成）。目前的做法并非如此——交易的某些部分被省略了。这就使得交易的某些部分具有延展性，即可以在以后进行更改。但是，这种可塑性并不包括支付的金额、发送人和接收人，这些都是以后无法更改的。我必须承认，我还没有深入研究这里的细节。据我所知，比特币开发者社区正在讨论这种延展性，并且正在努力减少或消除这种延展性。

具有多个输入和输出的交易

在上一节中，我介绍了单输入和单输出的交易是如何工作的。在实践中，创建多输入或多输出的比特币交易往往非常方便。下面我将谈谈为什么这很有用。但首先让我们来看看一个实际交易的数据：

```
1 {"hash":"993830...",
2  "ver":1,
3  "vin_sz":3,
4  "vout_sz":2,
5  "lock_time":0,
6  "size":552,
7  "in":[
8    {"prev_out":{"
9      "hash":"3beabc...",
10     "n":0},
11     "scriptSig":"304402... 04c7d2..."},
12    {"prev_out":{"
13      "hash":"fdae9b...",
14      "n":0},
15     "scriptSig":"304502... 026e15..."},
16    {"prev_out":{"
17      "hash":"20c86b...",
18      "n":1},
19     "scriptSig":"304402... 038a52..."}]},
20 "out":[
21   {"value":"0.01068000",
22     "scriptPubKey":"OP_DUP OP_HASH160 e8c306... OP_EQUALVERIFY OP_CHECKSIG"},
23   {"value":"4.00000000",
24     "scriptPubKey":"OP_DUP OP_HASH160 d644e3... OP_EQUALVERIFY OP_CHECKSIG"}]}
```

让我们逐行查看数据。这与单输入单输出事务非常相似，所以我会很快完成。

第1行包含交易其余部分的哈希值。这是用作交易的标识符。

第2行告诉我们这是比特币协议版本1的交易。

第3行和第4行告诉我们交易有三个输入和两个输出。

第5行包含`lock_time`。与单输入单输出情况一样，这被设置为0，这意味着交易立即最终确定。

第6行告诉我们交易的大小（以字节为单位）。

第7行到第19行定义了交易的输入列表。每个输入对应于先前比特币交易的一个输出。

第一个输入在第8行到第11行中定义。

特别是，第8行到第10行告诉我们输入将从具有`hash 3beabc...`的交易的`n=0`输出中取出。第11行包含发送比特币的人的签名，后面是一个空格，然后是发送者的公钥。

第12行到第15行定义了第二个输入，格式与第8行到第11行类似。第16行到第19行定义了第三个输入。

第20行到第24行定义了交易的两个输出列表。

第 21 行和第 22 行定义了第一个输出。第 21 行告诉我们输出值，0.01068000 比特币。和前面一样，第 22 行是 Bitcoin 脚本语言中的一个表达式。这里需要注意的是，`e8c30622...` 字符串是收款人的比特币地址。

第二个输出在第23行和第24行中定义，格式与第一个输出类似。

在这个描述中一个明显的奇怪之处是，尽管每个输出都有一个与之相关的比特币价值，但输入没有。当然，各自输入的价值可以通过查阅早期交易中的相应输出来找到。在标准的比特币交易中，交易中的所有输入的总和必须至少与所有输出的总和一样多。（唯一的例外是创世区块和coinbase交易，这两种交易都增加了比特币的总供应量。）如果输入的总和超过输出，那么多余的部分用作交易费。这将支付给成功验证包含当前交易的区块的矿工。

这就是多输入多输出交易的全部内容！它们是单输入单输出交易的一个相当简单的变体。

多入多出交易的一个很好的应用就是找零。例如，假设我想给你发送 0.15 个比特币。我可以用之前收到 0.2 个比特币的交易中的钱来给你汇款。当然，我不想把 0.2 个比特币全部寄给你。解决的办法是向你发送 0.15 个比特币，并向我拥有的比特币地址发送 0.05 个比特币。这 0.05 个比特币就是零钱。当然，这和你在商店里收到的零钱有点不同，因为这里的零钱是你自己付的钱。但大致的想法是相似的。

结论

这就是对比特币主要思想的基本描述。当然，我省略了很多细节——这不是一个正式规范。但我已经描述了比特币最常见的使用情况背后的主要思想。

虽然比特币的规则简单易懂，但这并不意味着理解规则的所有后果也很容易。关于比特币，还有很多东西可以说，我将在以后的文章中探讨其中的一些问题。

不过，现在我还是要先说说几个悬而未决的问题。

****比特币有多匿名？****许多人声称比特币可以匿名使用。这种说法导致了丝绸之路（及各种后继者）等市场的形成，这些市场专门经营非法商品。然而，比特币是匿名的说法只是一个神话。区块链是公开的，这意味着任何人都可能看到每一笔比特币交易。虽然比特币地址不会立即与现实世界中的身

份联系起来，但计算机科学家已经做了大量工作，研究如何使 "匿名 "社交网络去匿名化。区块链是这些技术的绝佳目标。如果在不久的将来，绝大多数比特币用户不能以相对较高的可信度被轻松识别出来，我会感到非常惊讶。可信度不会高到足以定罪，但会高到足以识别可能的目标。此外，身份识别将是追溯性的，也就是说，2011 年在丝绸之路上购买毒品的人，到 2020 年仍然可以根据区块链进行身份识别。这些去匿名化技术对计算机科学家来说是众所周知的，因此我们推测，对美国国家安全局来说也是如此。如果美国国家安全局和其他机构已经对许多用户进行了去匿名化处理，我一点也不会感到惊讶。事实上，具有讽刺意味的是，比特币经常被吹捧为匿名的。事实并非如此。相反，比特币可能是世界上有史以来最公开、最透明的金融工具。

****你能用比特币致富吗？****也许吧。蒂姆-奥莱利曾经说过"钱就像汽车里的汽油，你需要注意，否则你就会倒在路边" "但美好的生活不是参观加油站！"对比特币感兴趣的大部分人的人生使命似乎就是找到一个真正的大加油站。我必须承认，我对此感到困惑。我认为，把比特币和其他加密货币看作是实现新形式集体行为的一种方式，才是更有趣、更令人愉快的。这在智力上令人着迷，提供了奇妙的创造可能性，具有社会价值，而且还能把一些钱存进银行。但如果银行里的钱是你最关心的问题，那么我相信其他策略更有可能取得成功。

****我忽略的细节：****虽然这篇文章已经描述了比特币背后的主要思想，但还有很多细节我没有提到。其中之一是协议中使用的一种节省空间的好方法，它基于一种叫做梅克尔树的数据结构。这是一个细节，但却是一个精彩的细节，如果你喜欢有趣的数据结构，值得一试。你可以在最初的比特币论文中得到一个概述。其次，我对比特币网络知之甚少，比如网络如何应对拒绝服务攻击，节点如何加入和离开网络等等。这是一个引人入胜的话题，但也是一个细节混乱的问题，所以我省略了。你可以在上面的一些链接中阅读更多相关内容。

本文中的比特币脚本，我将比特币解释为一种数字网络货币。但这只是一个更大、更有趣的故事的一小部分。正如我们所看到的，每一笔比特币交易都与比特币编程语言中的脚本相关联。我们在这篇文章中看到的脚本描述的是 "爱丽丝给了鲍勃 10 个比特币 "这样简单的交易。但脚本语言也可以用来表达复杂得多的交易。换一种说法，比特币就是可编程货币。在后面的文章中，我将解释脚本系统，以及如何使用比特币脚本作为平台，尝试各种神奇的金融工具。

Thanks for reading. Enjoy the essay? You can tip me with Bitcoin (!) at address: 17ukKt1bNLAqdJ1QQv8v9Askr6vy3MzTZ. You may also enjoy the [first chapter](#) of my forthcoming book on neural networks and deep learning, and may wish to [follow me on Twitter](#).